Compresr SDK · TypeScript

LangGraph Integration

Compress graph state, checkpoints, and stores in LangGraph.js with drop-in, query-aware primitives. Cut token spend and storage size without rewriting your graph.

1 Install

```
npm install @compresr/sdk
export COMPRESR_API_KEY=cmp_...
```

Peers (`@langchain/langgraph` , `@langchain/langgraph-checkpoint` , `@langchain/core`) are optional — install only what you use. Node ≥ 20, dual ESM+CJS, typed end-to-end. Import from the dedicated entrypoint:

```
import {
  makeCompresrNode, CompresrCheckpointSerializer, CompresrStore,
} from '@compresr/sdk/integrations/langgraph';
```

A `compresrHandoffTool` is also available for supervisor / multi-agent patterns — see the SDK reference.

2 Compression node

`makeCompresrNode` returns an async graph node that reads a state field, compresses it against a query, and returns a patch. No-ops when input is missing, too short, or unchanged.

```
const compress = makeCompresrNode<typeof State.State>({
  apiKey:                process.env.COMPRESR_API_KEY!,
  contextKey:            'retrievedText',
  queryKey:              'question',
  targetCompressionRatio: 0.5,
  minTokens:             200,
});
graph.addNode('compress', compress);
```

3 Compressed checkpoints

`CompresrCheckpointSerializer` plugs into the `serde` slot of any LangGraph.js checkpointer (`MemorySaver` , `SqliteSaver` , `PostgresSaver`). Long strings on the listed fields are compressed on write. **Compression is lossy and one-way** — reads return the compressed string, not the original. Use the `fields` allow-list to scope compression to retrieved text, transcripts, and other paraphrasable payloads; leave titles, IDs, and anything you need to read back verbatim out of the list.

```
const checkpointer = new MemorySaver(
  new CompresrCheckpointSerializer({
    apiKey:                process.env.COMPRESR_API_KEY!,
    fields:                new Set(['retrievedText', 'transcript']),
    minTokens:             500,
    targetCompressionRatio: 0.5,
  }),
);
const graph = builder.compile({ checkpointer });
```

Without `fields` , every string ≥ `minTokens` anywhere in the state tree is compressed (including IDs and titles that exceed the threshold) — set the allow-list in any production deployment.

4 Compressed store

`CompresrStore` wraps any LangGraph.js `BaseStore` by composition: reads and namespace operations pass through; `put` and `batch` compress allow-listed string fields above the token threshold.

```
const store = new CompresrStore(new InMemoryStore(), {
  apiKey:    process.env.COMPRESR_API_KEY!,
  fields:    new Set(['retrievedText']),
  minTokens: 500,
});
await store.put(['users', 'u_42', 'memories'], 'memo_01', {
  retrievedText: '<3KB retrieval output>',
  score:         0.91,
});
```

5 Key parameters

Option	Default	Description
`apiKey` / `client`	—	API key, or a pre-built `CompressionClient` . One required.
`contextKey`	—	*(node only)* State key holding the text to compress. Required.
`fields`	—	*(serializer / store)* Allow-list of parent keys to compress. Strongly recommended.
`compressionModel`	`latte_v1`	`latte_v2` is the faster variant.
`targetCompressionRatio`	`0.5`	$[0, 1]$ = fraction of tokens to remove; > 1 = Nx factor.

Option	Default	Description
`minTokens`	node: `200` serializer / store: `500`	Skip compression below this threshold. Storage paths use a higher default to avoid compressing short rows.
`coarse`	(server)	`true` paragraph-level, `false` token-level.
`query` / `queryKey` / `queryExtractor`	—	Choose the query that drives compression: static string, state key, or extractor function. Highest-priority non-empty wins.
`onError`	`passthrough`	On transport, validation, or unexpected errors: return the original string so the graph keeps running. Use `raise` to surface exceptions.